

Table 1: Subset of GMF, Sirius, and JointJS Features Identified

Name	Definition	Role in the Editor
GMF:		
Canvas Mapping	Associates the diagram canvas with a semantic root element.	Defines the domain element that owns the diagram and contains top-level nodes.
Node Reference (Top/Child)	Specifies how nodes are retrieved from semantic containment references.	Determines which model elements appear at the root or inside containers.
Node Mapping	Defines how a domain element (EClass) is represented as a diagram node.	Binds semantics to visual figures, tools, and compartments.
Compartment Mapping	Defines a sub-region within a node for displaying contained elements.	Supports structured internal organization of nodes.
Label Mapping	Specifies how textual labels are derived from semantic elements.	Displays names or properties on nodes or edges.
Feature Label Mapping	Specialized label mapping bound to a specific structural feature.	Automatically renders concrete domain values (e.g., names).
Figure (Descriptor)	Reusable graphical definition serving as a visual template.	Encapsulates complex or nested figure structures.
Shape	Concrete graphical figure defining geometric appearance.	Provides visual body, including size, color, and layout behavior.
Label (Figure)	Figure type responsible for rendering text.	Displays human-readable information within or around elements.
LayoutData	Layout constraint attached to a figure.	Specifies positioning and sizing within a parent layout.
Rectangle	Concrete shape defining a rectangular graphical area.	Commonly used as the primary container or node body.
Sirius:		
Diagram Description	Defines a diagram type by specifying its root semantic element and associated mappings.	Declares a diagram view for visualizing domain models.
Node Mapping	Defines how semantic elements are represented as diagram nodes.	Controls node appearance and conditions under which elements are displayed.
Container Mapping	Defines nodes that can visually contain other nodes.	Represents grouping or hosting elements within the diagram.
Children Presentation	Specifies how child elements are arranged within a container.	Controls layout of contained elements (e.g., free, list, structured).
Style	Defines visual properties of diagram elements.	Determines shape, colors, size, borders, and label placement.
Label Expression	Expression computing the text shown for a diagram element.	Displays meaningful information derived from the semantic model.
JointJS:		

Name	Definition	Role in the Editor
Graph	Central in-memory model storing diagram elements and links independently of rendering.	Serves as the authoritative data structure for creation, modification, validation, and serialization.
Paper	Rendering canvas that visualizes a Graph as an interactive SVG/HTML diagram.	Manages display, user interaction, and synchronization with the underlying Graph.
Element (dia.Element)	Represents a diagram node with identity, position, size, and visual structure.	Defines nodes that can be styled, embedded, and connected via links.
Namespace	Identifies the shape definition space of an element.	Ensures correct resolution of visual classes across shape libraries.
Size	Defines the bounding box dimensions of an element.	Controls overall node dimensions and layout constraints.
Attrs	Declarative mapping assigning visual and semantic attributes to markup sub-elements.	Binds styling, layout, and data to specific visual components.
Object in Attrs	Selector-specific attribute configuration within <i>attrs</i> .	Configures individual visual parts (e.g., labels, compartments).
Markup	Declarative definition of an element's internal SVG/HTML structure.	Defines composition of graphical primitives forming complex shapes.
TagName	Specifies the SVG/HTML primitive used in markup (e.g., rect , text).	Determines low-level geometry of visual components.
Selector	Logical identifier assigned to a markup sub-element.	Enables targeted styling, positioning, and dynamic updates via <i>attrs</i> .
Rules	Behavioral policies governing element interaction (e.g., embedding).	Enforces containment and interaction constraints.
Functions inside Rules	Custom logic evaluated during user interaction.	Dynamically validates operations such as embedding or placement.
refX, refY, refWidth, refHeight	Relative positioning and sizing properties for sub-elements.	Enable proportional layout of internal visual components.

Table 2: Subset of GMF–Sirius and GMF–JointJS Correspondences

Source Feature	Target Feature	Rationale	Type
GMF–Sirius:			
Canvas Mapping	Diagram Description	Both define the diagram root and establish the semantic entry point of the editor. Despite differing mechanisms, their conceptual role in defining diagram context is equivalent.	Diagram Infrastructure – Structural

Source Feature	Target Feature	Rationale	Type
Node Reference (Top/Child)	Node Mapping / Container Mapping	Node References bind semantic containment to graphical elements. Depending on structural context (presence of compartments), the mapping yields either a Node Mapping or a Container Mapping in Sirius to preserve containment semantics and hierarchy.	Node Definition – Structural
Compartment Mapping	Container Mapping	Both define nested graphical regions for hosting child elements. Since Sirius does not separate compartments from containers, Container Mapping preserves the intended structural grouping.	Node Definition – Structural
LayoutData (XYLayout-Data)	Children Presentation	Although GMF uses absolute positioning and Sirius declarative layout policies, both regulate the organization of child elements within containers. Children Presentation approximates the layout intent encoded in GMF.	Layout Semantics – Visual
Feature Label Mapping	Label Expression	Both transform semantic attributes into textual labels. However, GMF allows multiple label mappings per node, whereas Sirius supports a single label expression, resulting in an approximate mapping.	Label Semantics – Visual
Figure	Container Mapping	Both contribute to structural composition of diagram elements. GMF supports arbitrary figure nesting, while Sirius separates structure and appearance, leading to an approximate correspondence requiring interpretation.	Layout Semantics – Visual
Shape	Style	Both determine visual rendering of diagram elements. Sirius Style abstracts geometric form and preserves the visual intent of GMF Shape definitions.	Figure Appearance – Visual
GMF– JointJS:			

Source feature	Fea- ture	Target feature	Fea- ture	Rationale	Type
Canvas	Map- ping	Graph, Paper		GMF Canvas Mapping defines the diagram root and global configuration. In JointJS, these responsibilities are separated: <i>Graph</i> manages the model, while <i>Paper</i> handles rendering and interaction. The correspondence preserves conceptual equivalence through structural decomposition.	Diagram Infrastructure – Structural
Node Mapping		Element (dia.Element), Namespace		Both introduce concrete node types tied to domain elements. In JointJS, identity and type resolution are handled through <i>Element</i> and its <i>namespace</i> , preserving the semantic role of GMF Node Mapping.	Node Definition – Structural
Compartment Mapping		tagName, selector, object in <i>attrs</i>		GMF compartments define nested visual regions. In JointJS, equivalent structure is realized by introducing sub-elements in markup and configuring them via selectors and <i>attrs</i> .	Node Definition – Structural
LayoutData (XYLayout-Data)		refX, refY, refWidth, refHeight		Both encode spatial intent of sub-elements within a container. GMF uses offset-based positioning, while JointJS expresses layout declaratively through relative references.	Layout Semantics – Visual
Size		width, height		Both specify node dimensions. GMF size constraints can be preserved as explicit width and height properties in JointJS elements.	Layout Semantics – Visual
Feature Label Mapping		tagName, selector, object in <i>attrs</i>		Both bind semantic attributes to textual labels. In JointJS, labels are defined as markup sub-elements and configured via selectors and <i>attrs</i> .	Label Semantics – Visual
Template Points (Poly-line)		d (SVG path data)		Both encode geometric path information. GMF template points define coordinate sequences, while JointJS represents equivalent geometry through SVG path commands.	Figure Appearance – Visual